

PointProgrammerNet: A Fixed-state additive FWP network for streaming point-cloud recognition

Dian Yu^{a,**}

^aUniversity of New South Wales, Sydney, NSW 2052, Australia

ABSTRACT

Most point-cloud networks obtain context by revisiting stored points, rebuilding neighborhoods, or applying a global maximum. These operations are effective offline but make a growing stream dependent on its history. PointProgrammerNet replaces point history with three fixed 128×170 matrices. For every observation, a normalized spatial address specifies where to write and a value vector specifies what to write; their outer product is added to each matrix. This simple construction makes independently encoded chunks exactly mergeable by matrix addition and also permits subtraction and exponential decay. Segmentation reads the matrices at a requested coordinate, while classification uses the matrices alone after all points have been discarded. We prove order invariance, chunk and distributed equivalence, fixed retained memory, and continuous dependence on incoming evidence. With XYZ input, the model obtains 92.95% point accuracy, 82.58% instance mIoU, and 79.10% category mIoU on ShapeNetPart, and 86.59% accuracy on ModelNet40. The three float32 states occupy 255 KiB regardless of stream length. At 8,192 accumulated points, updating one new chunk is $6.06\times$ faster than rebuilding the complete state, while the relative state error remains below 10^{-6} . The results show the accuracy cost and operational benefit of replacing an expanding point history with a bounded state that can be updated and combined directly.

© 2026 Elsevier Ltd. All rights reserved.

Keywords: point clouds ; streaming learning ; additive state ; fast-weight programmers ; segmentation ; classification

1. Introduction

A point cloud is an unordered set, but a sensor observes it over time. Processing within a frame should be independent of point order, and processing across frames should not require replaying the full history. PointNet Qi et al. (2017a) obtains a global descriptor by symmetric maximum pooling, while PointNet++ Qi et al. (2017b) builds multiscale metric neighborhoods. Both are effective offline. However, a maximum is not an additive state: fixed summaries cannot generally support deletion, independent chunk merging, and continuously weighted evidence by simple addition.

We study a stricter setting in which a point may be discarded immediately after it is written. Later computation accesses

only fixed matrices, plus a query coordinate for segmentation. A learned key distributes each point across state rows, and a second-order value records mass, position, learned content, and cross moments. Summing these per-point writes produces a fixed-size state.

Independent chunks can therefore be merged, a stored subset can be subtracted, and old evidence can be decayed. New points update the state without a pass over earlier observations. The same algebra supports distributed sensing: edge devices encode disjoint regions or time windows, transmit fixed matrices, and a central node adds those matrices before classification or segmentation. Segmentation reads a coordinate-conditioned field and stores no historical point features. Classification reads only state rows and discards the complete point list.

Our contributions are:

- a three-scale second-order fast-weight-programmer (FWP) state whose memory does not grow with the number of observed points;

^{**}Corresponding author

e-mail: dian.yu2@student.unsw.edu.au (Dian Yu)

URL: <https://orcid.org/0009-0002-0107-1014> (Dian Yu)

- additive point writes that support exact streaming, hierarchical merging, and distributed aggregation without re-playing chunk contents;
- a coordinate-conditioned segmentation head and a state-only classification head, neither of which pools over decoded points;
- proofs and experiments covering additivity, state operations, accuracy, and update cost.

Our compact PointNet++ implementations are controlled base-lines rather than claims of canonical reproduction. We use them to measure the accuracy cost of the fixed-state constraint.

2. Related Work

PointNet applies pointwise transforms followed by maximum pooling Qi et al. (2017a), and PointNet++ adds hierarchical metric neighborhoods Qi et al. (2017b). Deep Sets studies permutation-invariant functions built from sums Zaheer et al. (2017), while Set Transformer uses attention and inducing points Lee et al. (2019). Kernelized attention can be evaluated recurrently Katharopoulos et al. (2020), and fast-weight programmers express memory updates as additive outer products or delta corrections Schlag et al. (2021). PointProgrammerNet adapts this idea to geometry: its fixed states store spatial and learned second-order statistics and remain usable after the input points are released. We evaluate classification on ModelNet40 Wu et al. (2015) and part segmentation on ShapeNetPart Yi et al. (2016), which is derived from ShapeNet Chang et al. (2015).

3. Method

Let $X = \{\mathbf{x}_i\}_{i=1}^N$, with normalized coordinates $\mathbf{x}_i \in \mathbb{R}^3$. PointProgrammerNet replaces the growing point history with three matrices $\mathbf{S}_s \in \mathbb{R}^{128 \times 170}$, where $s \in \{f, m, b\}$ denotes branches initialized at radii 0.08, 0.20, and 0.60. The radii, spatial centers, point encoders, and read networks are learned independently. The state is created for each object or stream and is separate from the trained model parameters.

All three branches have the same state dimensions and use the same equations. Their different initial radii only provide distinct starting conditions; the trained network decides what each branch represents. A branch first maps a coordinate to learned content with a two-layer point network,

$$\mathbf{g}_s(\mathbf{x}) = \text{ReLU}(\text{BN}(W_{s2} \text{ReLU}(\text{BN}(W_{s1}\mathbf{x} + \mathbf{b}_{s1})) + \mathbf{b}_{s2})) \in \mathbb{R}^{32}. \quad (1)$$

The matrices W_{s1} , W_{s2} and biases are learned separately in each branch, and both hidden layers have width 32. Consequently, equal state sizes do not imply repeated features. This nonlinear map is evaluated independently for each point. It therefore enriches what a point writes without coupling that write to any other point; additivity is preserved because only the subsequent outer products are accumulated in the persistent state.

Table 1. The value written by one point. Every state row has these 170 columns.

Block	Dim.	Information retained
1	1	accumulated mass
$\mathbf{x}$	3	coordinate sum
$\mathbf{g}_s$	32	learned-content sum
$\mathbf{x}\mathbf{g}_s^\top$	96	position-content relation
$\mathbf{x}\mathbf{x}^\top$	6	geometric second moment
$\mathbf{g}_s \odot \mathbf{g}_s$	32	feature second moment

3.1. Additive state construction

Each of the 128 state rows has a learned center $\mathbf{c}_{sj} \in \mathbb{R}^3$. These centers are not sampled from an input cloud. Before training, a Sobol sequence simply places 128 initial anchors across $[-0.9, 0.9]^3$ so that the rows begin with broad spatial coverage. Every branch receives its own trainable copy, and training may move the anchors through the bounded parameterization $\mathbf{c}_{sj} = 0.95 \tanh(\boldsymbol{\theta}_{sj})$. For one observed coordinate $\mathbf{x}$, the branch measures its distance to all 128 anchors and converts those distances into 128 continuous write fractions. The positive radius $r_s = \exp(-\lambda_s)$ controls how concentrated these fractions are. The normalized address is

$$k_{sj}(\mathbf{x}) = \frac{\exp[-\|\mathbf{x} - \mathbf{c}_{sj}\|^2 / (2r_s^2)]}{\sum_{u=1}^{128} \exp[-\|\mathbf{x} - \mathbf{c}_{su}\|^2 / (2r_s^2)]}. \quad (2)$$

The denominator fixes every point’s total write mass at one. This prevents dense chunks from changing the scale of an individual write and lets the radius control concentration rather than total magnitude. In practical terms, the centers act like 128 soft spatial bins. A point near several centers gives those rows large fractions and all other rows small fractions; it is never assigned to only one bin. The address determines where the point contributes, while the following 170-dimensional value determines what statistics it contributes:

$$\mathbf{v}_s(\mathbf{x}) = [1, \mathbf{x}, \mathbf{g}_s, \text{vec}(\mathbf{x}\mathbf{g}_s^\top), x^2, y^2, z^2, xy, xz, yz, \mathbf{g}_s \odot \mathbf{g}_s]. \quad (3)$$

The blocks have dimensions $1 + 3 + 32 + 96 + 6 + 32 = 170$ and retain mass, first moments, learned content, coordinate-content products, and second moments. The leading one later supplies the denominator for local averages. The six quadratic coordinate terms and 32 squared feature terms retain spread that cannot be recovered from a mean. A point changes state row j by

$$[\Delta \mathbf{S}_s(\mathbf{x})]_{j,:} = k_{sj}(\mathbf{x}) \mathbf{v}_s(\mathbf{x})^\top. \quad (4)$$

This rank-one update uses the defining FWP operation: the key selects where to write, the value supplies what to write, and independent writes superpose by addition Schlag et al. (2021). PointProgrammerNet preserves this linear superposition, without a write normalization over the observed point set or a nonlinear overwrite of the previous state. Thus a row is a weighted accumulator rather than a slot for one selected point, and summing the outer products gives

$$\mathbf{S}_s(X) = \sum_{i=1}^N \mathbf{k}_s(\mathbf{x}_i) \mathbf{v}_s(\mathbf{x}_i)^\top = \mathbf{K}_s^\top \mathbf{V}_s. \quad (5)$$

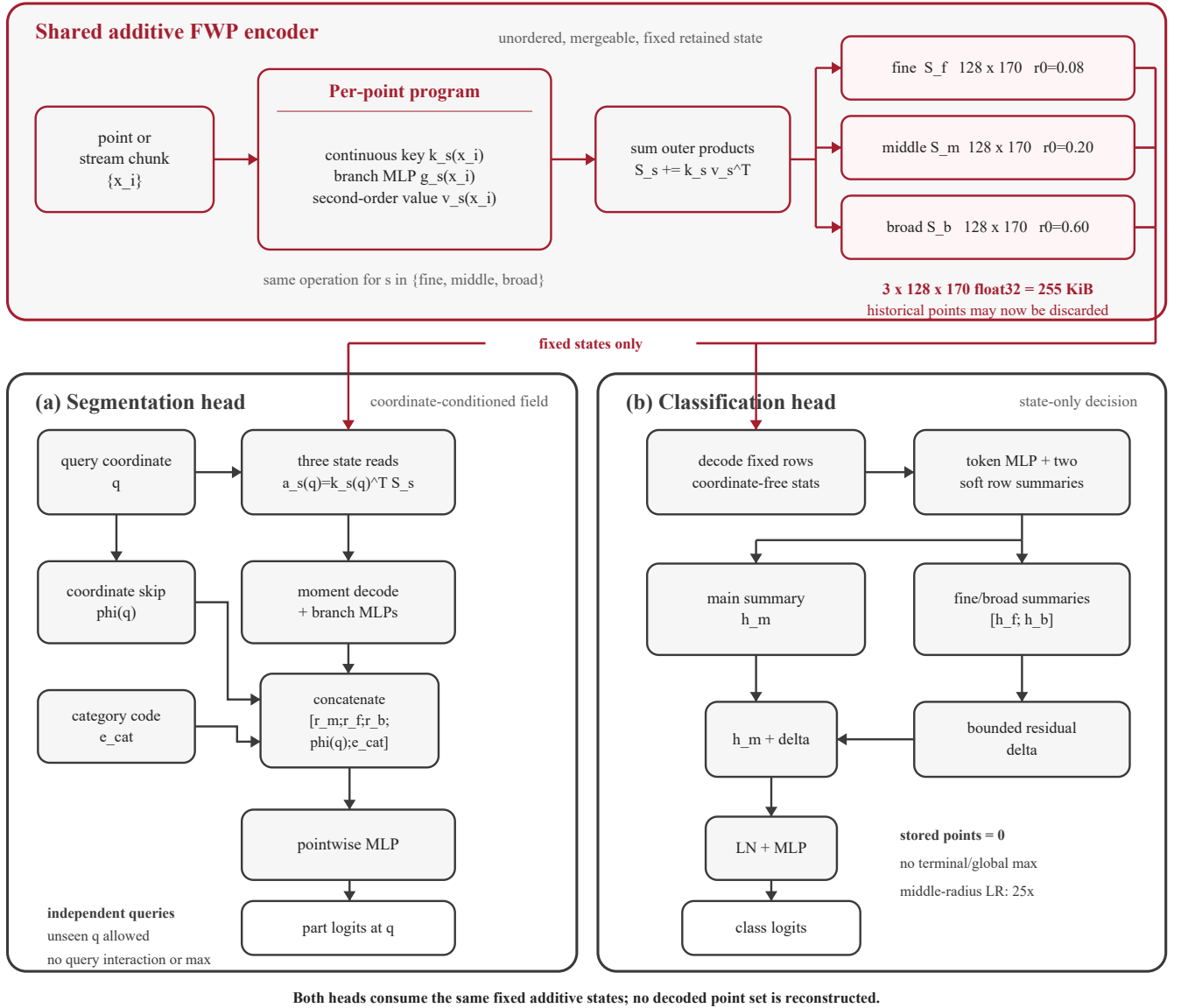


Fig. 1. PointProgrammerNet architecture. Red paths carry the three equal-size additive FWP states after historical points have been discarded. The segmentation head reads each state at query coordinate q , then concatenates the decoded context, coordinate skip, and category code. The classification head decodes fixed rows and adds a bounded fine–broad residual to the middle-state summary. Neither head reconstructs or pools over a decoded point set.

No point is assigned to a single row and no point is compared with another point. Once its three outer products have been added, it may be released.

For a newly arrived chunk, $K_s \in \mathbb{R}^{M \times 128}$ stacks its addresses and $V_s \in \mathbb{R}^{M \times 170}$ stacks its values. The chunk update is one matrix product $\Delta S_s = K_s^T V_s$. The persistent state is updated by $S_s \leftarrow S_s + \Delta S_s$, after which K_s , V_s , and the chunk points can be discarded. Chunk size changes temporary working memory, but not the retained representation.

The complete streaming workflow is therefore short. For each arriving chunk, the encoder (i) normalizes its coordinates in the shared reference frame, (ii) computes the three address and value matrices, (iii) adds the three products $K_s^T V_s$ to the persistent states, and (iv) releases all chunk-dependent tensors. Independent devices perform the same four steps and transmit

only their three matrices. At inference, the segmentation head evaluates only requested coordinates, whereas the classification head consumes the matrices directly. No later stage needs to regenerate a point list from the state.

3.2. Task readouts

For segmentation, x_i denotes an observed point that has already been added to the state and may then be discarded; q is only the coordinate at which a label is requested. On ShapeNet-Part, queries use the labeled input coordinates, but the decoder receives the coordinate and fixed state rather than the stored point feature. A query produces an address by Equation (2) and

reads

$$\mathbf{a}_s(\mathbf{q}) = \mathbf{k}_s(\mathbf{q})^\top \mathbf{S}_s = \sum_i \underbrace{\mathbf{k}_s(\mathbf{q})^\top \mathbf{k}_s(\mathbf{x}_i)}_{w_{si}(\mathbf{q})} \mathbf{v}_s(\mathbf{x}_i)^\top. \quad (6)$$

The coefficient $w_{si}(\mathbf{q})$ is an algebraic interpretation of the matrix read: it is large when point i wrote strongly to rows that $\mathbf{q}$ reads strongly. The implementation evaluates the left-hand matrix product directly; it neither reconstructs or revisits point i nor applies a global maximum over decoded points. Dividing the stored coordinate, feature, cross-product, and square sums by the read mass yields local coordinate and feature means $\bar{\mathbf{x}}_s, \bar{\mathbf{g}}_s$, cross moment $E_{xg,s}$, geometric moment $E_{xx,s}$, and feature energy $E_{gg,s}$. They form the 170-dimensional descriptor

$$\mathbf{d}_s(\mathbf{q}) = [\bar{\mathbf{g}}_s, \bar{\mathbf{x}}_s - \mathbf{q}, \log \tilde{\mu}_s, \text{vec}(E_{xg,s} - \mathbf{q} \bar{\mathbf{g}}_s^\top), \text{vech}(E_{xx,s} - \bar{\mathbf{x}}_s \bar{\mathbf{x}}_s^\top), \sqrt{[E_{gg,s} - \bar{\mathbf{g}}_s \odot \bar{\mathbf{g}}_s]_+ + 10^{-8}}]. \quad (7)$$

Here $\tilde{\mu}_s$ is the read mass with a positive numerical floor, vech keeps the six distinct entries of a symmetric 3×3 matrix, and $[\cdot]_+$ clips roundoff-induced negative variances. The descriptor deliberately combines complementary statistics. The feature mean supplies local semantic content, the relative centroid and geometric covariance describe local position and shape, and the log mass records how much evidence supports the read. The coordinate–feature term preserves how content changes around the query, while the feature standard deviation exposes mixtures and boundary variation that a mean alone would hide. Each branch applies the same $170 \rightarrow 160 \rightarrow 160$ BatchNorm–ReLU decoder. The final pointwise head is

$$\mathbf{z}_{\text{seg}}(\mathbf{q}) = h_{\text{seg}}([\mathbf{r}_f(\mathbf{q}); \mathbf{r}_m(\mathbf{q}); \mathbf{r}_b(\mathbf{q}); \phi(\mathbf{q}); \mathbf{e}_{\text{cat}}]), \quad (8)$$

where $\mathbf{r}_s \in \mathbb{R}^{160}$ is a decoded state read, $\phi(\mathbf{q}) \in \mathbb{R}^{32}$ is a coordinate feature, and $\mathbf{e}_{\text{cat}} \in \mathbb{R}^{16}$ is the ShapeNetPart category code. The coordinate path is a U-Net-like skip around state compression. The head acts on each query separately and never pools over decoded points. Historical features can be deleted; only coordinates that will later be labelled must remain. The same fixed state can also be queried at an unobserved coordinate.

For classification, no coordinates are retained. Each state row is divided by its stored mass to recover feature mean, centroid, relative density, coordinate covariance, feature standard deviation, and coordinate–feature covariance. These blocks again total 170 dimensions. An independent $170 \rightarrow 128 \rightarrow 128$ LayerNorm–GELU network encodes every row, giving $\mathbf{T}_s \in \mathbb{R}^{128 \times 128}$. Two learned score columns summarize the fixed rows:

$$\mathbf{U}_s = \text{softmax}_{\text{rows}}(\text{score}_s(\mathbf{T}_s)), \quad \mathbf{h}_s = H_s(\text{vec}(\mathbf{U}_s^\top \mathbf{T}_s)) \in \mathbb{R}^{256}. \quad (9)$$

This softmax is over 128 retained state rows, not over the input points, so its cost is fixed after encoding. The middle summary is the reference and the other two form a bounded residual:

$$\begin{aligned} \delta &= \sigma(a) \tanh(W[\mathbf{h}_f; \mathbf{h}_b] + \mathbf{b}), \\ \mathbf{z}_{\text{cls}} &= h_{\text{cls}}(\text{LN}(\mathbf{h}_m + \delta)). \end{aligned} \quad (10)$$

The residual projection starts at zero and $\sigma(a)$ starts at 0.1. The classifier uses only the fixed matrices and has no terminal point

maximum. Exact normalization equations, layer dimensions, and numerical safeguards are given in the supplementary material. Two score columns provide complementary fixed-cost views of each state instead of collapsing every row through one average. Using the middle branch as an initially stable reference and admitting the fine and broad summaries through a zero-started bounded residual prevents the three branches from perturbing one another before each has learned a useful summary; the residual can then grow when the auxiliary evidence improves classification.

3.3. Streaming properties

For arbitrary point multisets A and B , Equation (5) gives

$$\mathbf{S}_s(A \uplus B) = \mathbf{S}_s(A) + \mathbf{S}_s(B). \quad (11)$$

Finite sums also make the state invariant to point order. Thus separately encoded chunks, devices, or branches of an aggregation tree can be merged by addition. If a removable subset state is available, it can be subtracted; exponential forgetting uses $\mathbf{S}_{s,t} = \rho \mathbf{S}_{s,t-1} + \mathbf{S}_s(X_t)$. Exact distributed equivalence requires common trained weights, coordinate normalization, and reference frame.

To make the distributed claim explicit, suppose device d observes point multiset $X^{(d)}$ and all devices use the same encoder. A central node forms

$$\mathbf{S}_s^{\text{global}} = \sum_{d=1}^D \mathbf{S}_s(X^{(d)}) = \mathbf{S}_s\left(\bigoplus_{d=1}^D X^{(d)}\right). \quad (12)$$

The equality follows by expanding both sides into the same sum of per-point outer products. It holds whether the partial states arrive simultaneously, asynchronously, or through a tree of intermediate aggregators. Classification then uses only the merged matrices. Segmentation additionally receives the coordinates to be queried and the object category code. Raw point features need not leave the sensing devices.

The same algebra gives reversible state operations when the required partial state is available. If Y is a known subset, then $\mathbf{S}_s(X \setminus Y) = \mathbf{S}_s(X) - \mathbf{S}_s(Y)$. This removes the aggregate contribution of Y without replaying X . Multiplication by $\rho \in [0, 1]$ before an append implements exponential forgetting. These statements concern the representation itself; subtraction cannot recover individual points from a state when their separate subset state was not retained.

Retained memory is $O(\sum_s R_s D_s)$ and is independent of the observed-point count. Encoding M new points costs $O(M \sum_s R_s D_s)$ and does not depend on history length. To describe a gradually arriving observation, scale its write by evidence strength $t \in [0, 1]$:

$$\mathbf{S}_s(t) = \mathbf{S}_s^{\text{old}} + t \mathbf{k}_s(\mathbf{x}) \mathbf{v}_s(\mathbf{x})^\top. \quad (13)$$

The state is affine in t . Gaussian addresses, softmax, normalized moment reads with a positive floor, affine layers, BatchNorm or LayerNorm, ReLU or GELU, and stabilized square roots are continuous. Their composition makes the classification logits and fixed-coordinate segmentation logits continuous

Table 2. ShapeNetPart segmentation with XYZ input. PointProgrammerNet never pools over decoded query points.

Model	Point acc. (%)	Ins. mIoU (%)	Cat. mIoU (%)	Params
Lightweight PointNet++	93.41 $\pm$ 0.05	83.88 $\pm$ 0.11	79.51 $\pm$ 0.24	573,106
PointProgrammerNet	92.95 $\pm$ 0.02	82.58 $\pm$ 0.15	79.10 $\pm$ 0.25	341,909

and piecewise smooth in t . The spatial address is also continuous in the query coordinate q , so a fixed state defines a continuous, piecewise-smooth segmentation field. This does not mean a hard argmax label changes continuously: it can switch when two logits cross. It also does not establish calibration or monotonic improvement as evidence arrives.

For the released configuration, $R_s = 128$ and $D_s = 170$ for three branches, so the persistent representation contains 65,280 numbers. In float32 this is 261,120 bytes (255 KiB) per stream. Temporary tensors scale with the arriving chunk and can be released immediately. A segmentation application may retain a small set of coordinates of interest, but those coordinates are external queries rather than part of the historical state. A classification application retains no coordinates at all.

The additive rule treats all frames as exchangeable evidence. When frame order itself should alter memory, a future extension can apply a frame-level fast-weight delta update Schlag et al. (2021):

$$\mathbf{S}_{s,t} = \rho \mathbf{S}_{s,t-1} + \eta_t \mathbf{K}_{s,t}^\top (\mathbf{V}_{s,t} - \mathbf{K}_{s,t} \mathbf{S}_{s,t-1}). \quad (14)$$

Here $\mathbf{K}_{s,t}$ and $\mathbf{V}_{s,t}$ package all keys and values in frame t , ρ retains old state, and η_t is an update rate. Reordering points within a frame multiplies both matrices on the left by the same permutation P ; the correction is unchanged because $P^\top P = I$. The extension therefore preserves within-frame point-order invariance while introducing ordered state transitions between frames. It is a proposed direction and is not used in the reported experiments because it intentionally gives up exact additivity across frames.

4. Experiments

4.1. Protocol and accuracy

We center each object and divide XYZ by its maximum absolute coordinate, using 1,024 points. ShapeNetPart contains 13,972 training and 2,874 test shapes; ModelNet40 contains 9,840 and 2,468. Models train for 20 epochs with AdamW, base learning rate 3×10^{-3} , weight decay 10^{-4} , cosine decay, and batch sizes 16 and 32. Classification uses 0.1 label smoothing. Results are mean $\pm$ population standard deviation over seeds 0, 1, and 2. The segmentation model contains 341,909 trainable parameters and the state-only classifier contains 529,426. In classification, the middle radius alone uses a 25 $\times$ learning-rate multiplier without weight decay; the controlled screen below motivates this robustness-oriented choice. All reported PointProgrammerNet variants use three 128×170 states and no global maximum over input or decoded points. Our compact PointNet++ implementations are controlled local baselines rather than claims of canonical reproduction.

Table 3. ModelNet40 classification with 1,024 points. Density denotes the fixed density-gradient shift.

Model	Clean (%)	Density (%)	Drop (pp)	Params
Lightweight PointNet++	89.84 $\pm$ 0.43	85.47 $\pm$ 0.88	4.38 $\pm$ 1.18	207,784
Wider PointNet	87.90 $\pm$ 0.27	87.55 $\pm$ 0.38	0.35 $\pm$ 0.12	442,838
PointProgrammerNet	86.59 $\pm$ 0.23	82.47 $\pm$ 0.93	4.12 $\pm$ 0.71	529,426

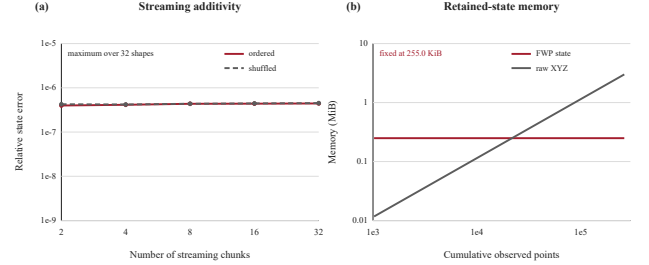


Fig. 2. Additive streaming and fixed retained memory. Ordered and shuffled chunk writes reproduce the single-batch FWP state up to floating-point accumulation error. The three 128×170 float32 states occupy 255 KiB regardless of stream length, whereas stored XYZ coordinates grow linearly with the number of observations.

PointProgrammerNet is 0.46 points lower in point accuracy and 1.30 points lower in instance mIoU than the lightweight PointNet++ control, while using 40.3% fewer parameters.

The state-only classifier reaches 86.59% clean accuracy, 3.25 points below lightweight PointNet++ and 1.31 points below wider PointNet. Its density-shift accuracy is 82.47%. This quantifies the accuracy cost of discarding the point list and classifying only from mergeable states.

4.2. Additivity and update cost

Across 32 objects, states built from 2–32 ordered or shuffled chunks agree with one-shot construction to relative error below 5.3×10^{-7} . Append, merge, removal, and decay tests remain below 9.8×10^{-7} . The three float32 states contain 65,280 scalars (255 KiB), independent of stream length. Raw XYZ is smaller for short clouds; the benefit is bounded retained memory and direct state operations over long or distributed streams.

On an RTX 3070 Ti Laptop GPU, four streams receive 256 new points per frame. Incremental latency stays near 1.46–1.53 ms, whereas reconstruction grows with history. Updating becomes 1.52 $\times$, 3.04 $\times$, and 6.06 $\times$ faster at 2,048, 4,096, and 8,192 accumulated points.

4.3. State algebra in practice

Append, independently encoded merge, stored-frame removal, and exponential decay were compared with direct reconstruction on 32 ShapeNetPart objects. Their mean relative errors were 2.11×10^{-7} , 2.13×10^{-7} , 3.03×10^{-7} , and 1.77×10^{-7} , respectively, and all maxima were below 9.8×10^{-7} . The remaining discrepancy is consistent with float32 accumulation order. After 16 frames, ordinary addition produced state norm 14.51; decay with $\rho = .9$ limited the norm to 7.42. This experiment validates the state equations and does not claim temporal-task accuracy.

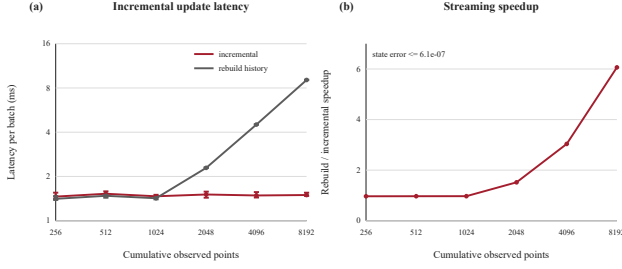


Fig. 3. FWP state-construction cost for device-resident point streams. (a) An incremental update processes one new 256-point frame, whereas reconstruction processes every accumulated point. Values are median wall-clock latency over seven interleaved trials of 30 repetitions for four streams; whiskers show the observed range. (b) The speedup grows with history length, while incremental and reconstructed states remain numerically equivalent.

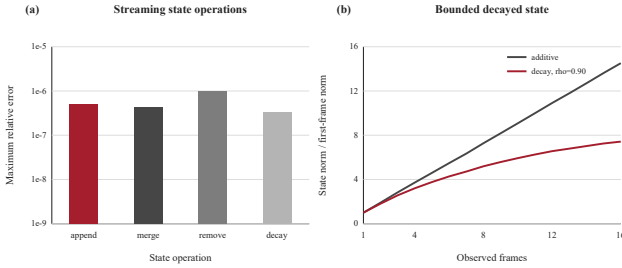


Fig. 4. Streaming state operations. (a) Maximum relative errors for append/merge, frame removal, and exponential decay, measured against independently reconstructed FWP states on 32 ShapeNetPart objects. (b) Ordinary addition increases the state magnitude with the number of frames, while exponential decay limits the influence of older frames without changing state size.

4.4. Why the final heads were retained

The screens isolate one architectural decision at a time. Equal $170 \rightarrow 160 \rightarrow 160$ segmentation decoders improve instance mIoU by 0.30 ± 0.30 points and remove 24,480 parameters. Distinct radius initialization improves instance mIoU by 0.74 ± 0.42 . Applying the same normalized address equation to all three branches improves instance and category mIoU by 0.35 ± 0.16 and 0.85 ± 0.37 over a mixed-key variant. For state-only classification, the middle-reference residual improves clean and density-shift accuracy by 2.67 ± 1.60 and 3.78 ± 1.71 over direct concatenation. Increasing only the middle-radius learning rate gives a variable clean-accuracy change but a consistent 0.91 ± 0.33 shifted-accuracy gain. These results motivate the released heads while keeping the state-writing rule symmetric across branches. Exact screen definitions and per-condition values are provided in the supplement.

5. Discussion and Limitations

The intended setting is a growing or distributed point stream. A sensor can encode a frame, merge its state, and release the frame. Several cameras, LiDARs, robots, or edge processors can encode disjoint spatial regions or time windows locally and send fixed states to a server. Addition then produces the same representation as centralized encoding, so aggregation can be

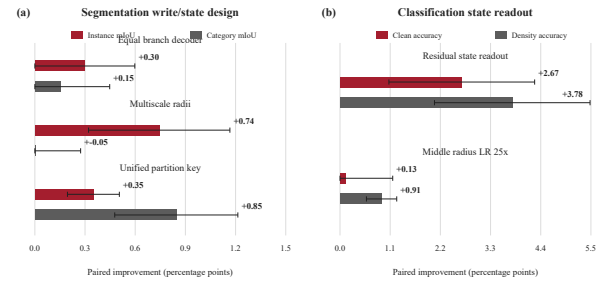


Fig. 5. Three-seed, five-epoch architecture screens. Bars show paired mean changes and whiskers show population standard deviations. Segmentation tests equal branch decoders, distinct radius initialization, and a common normalized spatial address. Classification tests residual versus concatenated branch summaries and the middle-radius learning rate.

asynchronous and hierarchical. Raw point sets need not leave the sensing devices. A segmentation service can retain only coordinates it plans to query, while a classifier updates its evidence without replaying earlier points. None of these operations compares a new point with all previously decoded points.

5.1. Worked distributed inference

Consider several LiDAR nodes observing different parts of a scene. The trained encoder is copied to every node, and all nodes express coordinates in a common frame with the same normalization. Node d encodes its local chunk into $\{\mathbf{S}_f^{(d)}, \mathbf{S}_m^{(d)}, \mathbf{S}_b^{(d)}\}$ and can then delete the local features. A relay may add any subset of these triples; the server adds the relay outputs and obtains Equation (12). The order of arrival and the shape of the communication tree do not affect the real-arithmetic result. Each transmitted triple has the same 255 KiB size whether the local chunk contains hundreds of points or represents a long accumulated stream.

For classification, the server passes the merged triple directly to the state-row readout. A new device or frame changes the class evidence by adding one more triple, without decoding and pooling the earlier clouds again. For segmentation, the server supplies a coordinate $\mathbf{q}$ to the three matrix reads. Queries may be the coordinates of selected observed points, a region requested by an operator, or positions on a regular grid. Only these coordinates need to be available at read time. Because Equation (6) depends smoothly on $\mathbf{q}$, nearby queries receive smoothly changing state evidence, including at coordinates that were not explicitly observed. The current experiments evaluate labels only at observed ShapeNetPart points, so accuracy at new coordinates remains an open empirical question.

The fixed states are summaries rather than compressed point archives. They are sufficient to evaluate the learned heads, but they are not designed to reconstruct every original observation. Two point sets can produce similar moment tables, especially when many spatial addresses overlap. The learned centers, radii, content networks, and second-order terms determine which distinctions survive this compression. This explains both the bounded-memory benefit and the remaining accuracy gap to a neighborhood method that retains or rebuilds detailed point relations.

The fixed state is an information bottleneck. PointNet++ forms neighborhoods around observed points and PointNet preserves channelwise extremes, whereas PointProgrammerNet retains finite moment tables. The state also exceeds bare-XYZ storage for short clouds; its benefit appears when bounded communication, merging, or long streams justify the fixed cost. Distributed equivalence requires identical preprocessing and a shared coordinate frame, and segmentation still reads the state once per requested coordinate. Evaluation is limited to XYZ input, one dataset per task, one timing GPU, and local controlled baselines. Unseen-query accuracy, temporal calibration, additional attributes, and delta adaptation remain future work.

6. Conclusion

PointProgrammerNet summarizes a point cloud with fixed additive matrices. Continuous keys and second-order values support unordered writes, exact distributed merging, removal, decay, and updates whose cost does not depend on history length. Coordinate-conditioned segmentation and state-only classification operate without pooling over decoded points. Accuracy remains below the hierarchical baselines, but the model provides a compact and testable design for centralized or distributed point-cloud inference from a bounded streaming state.

CRedit authorship contribution statement

Dian Yu: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Visualization, Writing—original draft, and Writing—review and editing.

Data availability

The code needed to reproduce the reported models is provided as supplementary material. ShapeNetPart and ModelNet40 are public datasets available from the original sources cited in this article.

References

- Chang, A.X., Funkhouser, T., Guibas, L., Hanrahan, P., Huang, Q., Li, Z., Savarese, S., Savva, M., Song, S., Su, H., Xiao, J., Yi, L., Yu, F., 2015. Shapenet: An information-rich 3d model repository. arXiv:1512.03012 .
- Katharopoulos, A., Vyas, A., Pappas, N., Fleuret, F., 2020. Transformers are RNNs: Fast autoregressive transformers with linear attention, in: ICML, pp. 5156–5165.
- Lee, J., Lee, Y., Kim, J., Kosiorek, A., Choi, S., Teh, Y.W., 2019. Set transformer: A framework for attention-based permutation-invariant neural networks, in: ICML, pp. 3744–3753.
- Qi, C.R., Su, H., Mo, K., Guibas, L.J., 2017a. Pointnet: Deep learning on point sets for 3d classification and segmentation, in: CVPR, pp. 652–660.
- Qi, C.R., Yi, L., Su, H., Guibas, L.J., 2017b. Pointnet++: Deep hierarchical feature learning on point sets in a metric space, in: NeurIPS, pp. 5099–5108.
- Schlag, I., Irie, K., Schmidhuber, J., 2021. Linear transformers are secretly fast weight programmers, in: ICML, pp. 9355–9366.
- Wu, Z., Song, S., Khosla, A., Yu, F., Zhang, L., Tang, X., Xiao, J., 2015. 3d shapenets: A deep representation for volumetric shapes, in: CVPR, pp. 1912–1920.
- Yi, L., Kim, V.G., Ceylan, D., Shen, I.C., Yan, M., Su, H., Lu, C., Huang, Q., Sheffer, A., Guibas, L., 2016. A scalable active framework for region annotation in 3d shape collections. ACM Transactions on Graphics 35, 210:1–210:12.

Zaheer, M., Kottur, S., Ravanbakhsh, S., Poczos, B., Salakhutdinov, R., Smola, A.J., 2017. Deep sets, in: NeurIPS, pp. 3391–3401.